DOS Protected Mode Services (DPMS) 1.0 API

OpenDOS Developer's Reference — Programmer's Guide

Caldera, Inc.

Archived HTML documentation converted for reading

Front Matter / Table of Contents

DOS Protected Mode Services

Table of Contents

Chapter 1 - DOS Protected Mode Services

Chapter 2 - DPMS Function Reference

Chapter 3 -Debugging DPMS Clients

Appendix A - DPMS Error Codes

info@caldera.com
Copyright © 1994, 1997 Caldera, Inc. All rights reserved.

Chapter 1 DOS Protected Mode Services

Introduction

The original IBM* PC standard, for which DOS became the standard operating system, was based around the 8088 processor which had a memory address space of one megabyte. Compatibility with this standard now limits the memory available in standard DOS systems. More and more demand has been put upon DOS systems since the original PC, and supporting this demand means that a typical DOS configuration today loads several device drivers and other resident system extensions, which all reduce the conventional memory available to standard DOS applications.

The 80286 and later processors descended from the 8088 and compatible 8086 provide an alternative mode of operation, called protected mode. Protected mode provides programs with access to much more memory space than the single megabyte of the 8088. The extra memory space provided is called extended memory. Unfortunately standard DOS programs cannot run in protected mode, and therefore cannot access extended memory directly. Many DOS applications have been adapted or written to run in protected mode, but there are memory and other overheads in doing this, so few resident system extensions run in protected mode.

DOS Protected Mode Services (DPMS) is a set of services that is designed for DOS resident system extensions to run in protected mode. DPMS provides all the benefits of protected mode operation without the headaches. DOS compatibility is not compromised, and it is easy to create resident system extensions that can adapt at initialization time to DPMS and non-DPMS configurations.

This document describes the DPMS 1.0 specification and issues raised by programming for DPMS using the DPMS 1.0 System Developers Kit (SDK). It is assumed that the reader is familiar with the Intel* x86 range of processors.

Protected Mode

It is beyond the scope of this manual to describe protected mode programming in full but this introduction is included so that programmers familiar with standard DOS programming, but not protected mode, can get an idea of what is required to program for DPMS. It also defines some terms used in the document.

Introduction to Protected Mode

In a standard DOS system, there are no restrictions upon what memory and other hardware a program can access. Therefore it is possible for a faulty or malicious piece of software to bring down the whole system. Protected mode, as the name implies, provides an operating

system with the processor hardware features necessary to have complete control over the memory and other hardware and software resources that programs can access.

For compatibility with earlier processors such as the 8088, processors that support protected mode can also run in a mode that is compatible with those processors, and this is called real mode. 386* and later processors also provide a third mode called virtual-8086 mode in which standard DOS programs can run in a semi-protected environment. For the purposes of this introduction virtual-8086 mode can be treated the same as real mode.

NOTE: The term real mode is used throughout this document to refer to both true real mode and virtual-8086 mode. DPMS clients running on a 386 or later typically run in virtual-8086 mode rather than real mode.

For most software, protected mode operation is very similar to real mode. The major difference lies in the way segments are defined and used. It is possible to write software that can operate in either mode but most standard DOS programs cannot run in protected mode. Therefore standard DOS systems must run in real or virtual-8086 mode.

It is possible to mix real and protected mode software in a system by switching modes. Transitions from real to protected mode are referred to as up, transitions in the opposite direction are down. A call-up is a call that originates in real mode, executes some code in protected mode, and returns to the real mode code. A call-down is the opposite.

Segments, Descriptors, and Selectors

Segments are defined and used differently in protected mode than they are in real mode. In real mode, segment registers contain a value which is directly related to the base address; all segments are 64 KB in size and can be used by any software for code or data without restriction. In protected mode, on the other hand, segment registers contain values which are used to locate entries in tables which describe the segments and how they may be used. The tables are in memory and have one entry, called a descriptor, per segment. Each descriptor consists of eight bytes which describe the segment's base address, size, and how it may be used.

Segment size is described by the maximum offset that may be accessed within the segment, which is the size minus one, and this is called the limit of the segment. Protected mode segments can be larger than 64 KB on 386 and later processors.

The values loaded into segment registers which act as offsets into descriptor tables are called selectors.

In protected mode, different segments must be used for code and data, and code segments cannot be written to.

There are two descriptor tables at any time: the Global Descriptor Table (GDT) and Local Descriptor Table (LDT). Normally the GDT would be fixed while there would be a different LDT for each task if in a multitasking system. Bit 2 of a selector determines whether the corresponding descriptor is in the GDT or LDT.

Descriptors and selectors are assigned privilege levels, from zero to three. Zero is the most privileged level and three is the least. Programs running at lesser privilege levels are prevented by the processor from loading segment registers with selectors for higher privilege segments, and so those higher level segments are protected.

Restrictions

· Code segments cannot be written to. This means that the CS:override prefix cannot be used for writes, a practice which is common in real mode programming. MASM and compatible assemblers provide a command line switch (/p) to check for such overrides.

· Memory can only be accessed via descriptors. For example, real mode software might access a byte in the PC BIOS data area by loading a segment register with 00040h, but in protected mode it must first set up a descriptor for the required segment and then load the corresponding selector.

· Segment arithmetic cannot be used. If real mode software overruns the end of a segment, it can access the memory following it by simply adding 01000h to the segment register. That cannot be done in protected mode as there is no relationship between the selector and base address values. Either the base address of the descriptor must be updated or a new descriptor must be set up to access the required memory.

Memory Mapping

386 and later processors support memory mapping while in protected and virtual-8086 modes. This means that any 4 KB physical block of memory can be mapped at any linear address within the entire 4 GB address space.

By convention, addresses that may be mapped are referred to as linear addresses even when they have not been mapped. The addresses of the underlying physical blocks are referred to as physical addresses.

Protected Mode Environments

Protected mode programming in general is not just a matter of using segments differently to real mode, but also involves the construction and management of an entire protected mode system, which is especially complicated if the protected mode system is based upon DOS. Many applications that have been written or adapted to run in protected mode use DOS extenders, which are sets of tools and libraries that enable standard DOS programs to be easily ported to protected mode operation. A DOS extender runs applications in a protected mode environment which emulates that of DOS so that, for example, a DOS function call (INT 021h instruction) in protected mode is transparently passed down to the real mode DOS.

If a system is already running in protected mode then it is impossible for any other piece of software to get control of the processor. Therefore in order for protected mode applications to run, a number of standard interfaces have developed which enable protected mode software to run in an existing protected environment. The main interfaces are the Virtual Control Program Interface (VCPI) and the DOS Protected Mode Interface (DPMI).

DPMS Features

The main functions of DPMS are to manage protected mode system resources and to handle transitions between real (or virtual-8086) and protected modes.

DPMS is not a DOS extender. Although it has a similar acronym and some similarities, it is not the same as DPMI. It is a minimal suite of services tailored to the needs of resident system extensions. It is possible to run such software in protected mode using one or more other interfaces, but DPMS is usually more appropriate because

· DPMS provides a consistent interface, independent of environment. It insulates its clients from environment changes caused when MS Windows runs.

· DPMS uses very little standard DOS memory.

· You can supply the DPMS server with your software for low royalty payments.

DPMS API functions include

· Calls from real to protected mode and vice-versa

· Descriptor management

· Memory management

The DPMS interface is fully re-entrant.

DPMS Clients

The following sections introduce client structure and initialization.

Client Structure

Clients are initially loaded and run as standard DOS programs, and can be either device drivers or TSRs or both. You can therefore use standard tools to create them.

Clients typically have three parts

· Initialization code and data which is discarded when the client becomes resident.

· Real mode code and data which is the minimum that must stay in real mode memory.

· Relocatable code and data which, if DPMS is present, is moved into extended memory and runs in protected mode. This code and data may use 386-specific features such as 32-bit segments, but normally it will not, so that it can run in real mode without DPMS or in protected mode using DPMS on 80286 processors.

Initialization

Client initialization includes the following steps:

· Find any DPMS server.

· Allocate protected mode descriptors.

· Allocate extended memory.

· Set the descriptors' base addresses, limits, and types.

· Relocate the code and data into extended memory.

· Fix up any calls in the real mode segment whose destination isin the code that now executes in protected mode. Replace them with callsto DPMS call-ups.

· Perform functions specific to particular clients, such asregistering as a device driver with DOS, hooking real mode interrupts,and initializing data structures.

· Free any real mode memory that is no longer required.

· Call DOS to terminate and stay resident.

The client is now resident, and either DOS or an interrupt can call it through entry points in the real mode portion. Because the client does its work in protected mode, the real mode portion may consist of little more than a DPMS call up to the protected mode portion.

In order to call any real mode function, for example to call DOS, the protected mode code must go via one of the DPMS call down functions. The call down sets all real mode registers, but any data that is passed in memory, for example, a buffer, must reside in memory that is accessible in real mode.

Some clients require DPMS, and fail to install if no server is present. Others can operate entirely in real mode if there is no DPMS server.

Overview of DPMS Functions

The following sections give an overview of the DPMS functions. Chapter 2, "DPMS Function Reference," contains detailed descriptions of all the DPMS functions.

Descriptor Management

Reuse descriptors where possible, because descriptor tables have a finite size, and relinquish them when no longer needed. Although the only way to address memory in

protected mode is via a descriptor, you must explicitly initialize each descriptor to refer to an allocated memory area before you can use it.

Clients should try to reuse the same descriptor to address different objects at different times, because it takes time to allocate descriptors and because there is a finite number of descriptors available.

Clients usually need to allocate and initialize descriptors before they enter protected mode. Use the DPMS_D_ALLOC, DPMS_D_ALIAS and DPMS_M_RELOC functions to allocate descriptors. Refer to the description of DPMS_D_ALLOC on page 2-39, DPMS_D_ALIAS on page 2-43, and DPMS_M_RELOC on page 2-66 in Chapter 2, "DPMS Function Reference." Descriptors allocated via DPMS_D_ALLOC must be initialized after allocation. Depending upon what the descriptor is to be used for, initialization is done in one of the following ways:

· Allocating linear memory via DPMS_M_ALLOC (see page 2-54) or otherwise, then calling DPMS_D_SET_BASE (see page 2-47), DPMS_D_SET_LIMIT(see page 2-48), and DPMS_D_SET_TYPE (see page 2-50) to set the descriptor to refer to that memory.

· Calling DPMS_D_ALIAS_REAL (see page 2-45) to create an alias to a real mode segment.

· Passing the descriptor to DPMS_M_RELOC (see page 2-66).

If the client terminates, it must explicitly free up descriptors using DPMS_D_FREE (see page 2-41) before terminating.

Memory Mapping Functions

On systems which implement linear-to-physical memory mapping, DPMS offers the functions DPMS_M_MAP and DPMS_M_UNMAP, which allow clients to map and unmap any physical memory, which may be non-contiguous, into DPMS linear address space. Refer to the description of DPMS_M_MAP on page 2-57, and DPMS_M_UNMAP on page 2-60 in Chapter 2, "DPMS Function Reference," for more details.

If these functions are supported, then clients must use them if they need to access extended memory other than that allocated via DPMS.

For example, consider a disk cache that must run both with and without DPMS. When running without DPMS, it would typically use XMS memory. When running with DPMS, it could use memory allocated via DPMS but this would mean that the memory interface would be completely different between the DPMS and non-DPMS installations, and would also make memory lending more difficult. The cache could still use XMS memory with DPMS, but this would mean that calls to the XMS driver to do block moves to and from the memory would involve mode transitions and would therefore be slower than without DPMS. Instead, the client could lock the XMS memory it had allocated, get the physical address(es) of that memory, and call DPMS_M_MAP to map the physical memory into DPMS linear address space. It could then access the memory directly instead of via XMS block moves, meaning that the cache could be faster with DPMS than without.

Extended memory allocated or mapped via DPMS may only be accessed by the DPMS client while it is in protected mode under DPMS. Access may fail if attempted via any other interface.

Mode Transitions

DPMS provides a number of functions for calling protected mode from real mode and vice-versa. These fall into two categories. The first set of functions take a parameter block which contains values for all registers to be used in the opposite mode. The parameter block has the same format for all calls. Refer to the description of the DPMS_CALL_PROT function on page 2-18, in Chapter 2, "DPMS Function Reference," for the structure of the parameter block and a full description of that type of function.

The parameter block includes CS and (E)IP register values. These are the addresses of the client procedure to be called. When the called procedure returns, using a RETF or an IRET, the return goes via DPMS which performs the inverse mode change, copies the new register values back into the parameter block, and returns to the original mode code.

Most calls from real or virtual-8086 to protected mode and vice-versa are to addresses that are determined at initialization time. Furthermore, many calls require parameters to be passed in registers that would have to be stored in a DPMS register structure, if calling via the call-up or -down functions already described. There is therefore an alternative, more convenient way of making inter-mode calls.

Using the alternative method, the client registers a default procedure to be called, together with default values for certain registers, including the segment registers. This is done by passing the address of a default register structure to the server. The server fills in part of the structure with the code necessary to effect the inter-mode call, and this code also contains a client ID. The client can register almost any number of default procedures in this way; each procedure is called via the code in the corresponding default register structure.

Refer to the description of the DPMS_REG_DEF_PROT function on page 2-29, in Chapter 2, "DPMS Function Reference," for the default register structure and a full description of that type of function.

Protected mode has several advantages over real mode, only one of which is the ability to execute code in extended memory. It is feasible to switch into protected mode to execute code within the first megabyte. If the code is quite small, you may not want to relocate it.

MS Windows 3.n Restrictions

When MS Windows starts up in either standard or enhanced mode, all DPMS services should continue normally. However, MS Windows normally allocates all available memory and replaces extended and expanded memory interfaces with its own. All memory allocations made while MS Windows is running are temporary and will disappear when MS Windows terminates. This applies both to memory allocated via the native extended or expanded memory interface and that allocated via DPMS, as the DPMS server normally calls down to the underlying memory manager to allocate the physical memory required.

To avoid this problem, clients should allocate all memory they need before Windows loads. If the client needs to co-operate with other programs, including MS Windows, that may occasionally want memory owned by the client, hook the memory manager interface and lend memory as appropriate.

API Implementation

Constants and structures for the DPMS API are defined symbolically in include files: DPMS.INC for assembly language and DPMS.H for the `C' language.

The only code you need to use DPMS is the code you write yourself. DPMS does not require a run-time library.

DPMS clients typically perform low-level functions, so the API is created primarily for assembly language. You can write parts of a client in a high level language, but use run-time library functions with caution as they may not be compatible with protected mode.

Sample Client Source Code

To help you understand what a DPMS client might look like, the SDK includes VDISK.ASM, which is the complete source code for a virtual disk driver that uses DPMS to relocate and execute much of its resident code.

This program has extensive comments to guide you through its operation. It uses all the main DPMS features. After installation when running under DPMS, less than 400 bytes of code and data remain in real mode memory.

The sample VDISK requires XMS to run.

Server Source Code

The SDK includes the full source code for the DPMS server (DPMS.EXE) to help you debug clients and to optimize the server for use on either 286 or 386 (and above) platforms. The following tools are required in order to build the server. Caldera do not supply these tools.

· Borland Turbo Assembler (TASM) v2.51
· Borland Turbo Link (TLINK) v5.1
· MS Macro Assembler v5.10b
· MS Linear Executable Linker (LINK386.EXE) v1.00.056

In the \SOURCE\DPMS directory, edit the MAKE.BAT files so that the paths point to the correct tools on your computer. Run MAKE to build the server (DPMS.EXE). This is copied into the \SOURCE\DPMS\BIN directory.

You need at least 600 Kb of TPA to build DPMS.EXE. We therefore recommend that you use DR-DOS because of the extra TPA available. You can build the server into the DR-DOS kernel if you need an embedded solution and the DR-DOS source code is available from Caldera's web site.

DPMSTEST

DPMSTEST is a test program for DPMS servers. It is included in the SDK so that developers of DPMS servers can test their servers. The program exercises every DPMS function and outputs the results. While it is fairly thorough, it is not guaranteed to find every possible fault.

The test program is straightforward to use. Enter DPMSTEST /? for a summary of its use.

DPMS Limitations

DPMS 1.0 has the following known behavior and limitations:

· DPMI, if present, is not available to DPMS clients in protected mode as protected mode INT 031h instructions are reflected to real mode.

· There is no API for hooking interrupts in protected mode. You must hook all hardware and software interrupts in real mode, and use call-ups to pass control to a protected mode handler if required.

· Most DPMS calls from real mode are processed by the server in protected mode, and therefore the server must switch modes twice for each call. These mode switches are not required for DPMS calls originating in protected mode. This gives a performance advantage for protected mode over real mode. It is therefore recommended that you make any frequent API calls, such as setting up descriptors, from protected mode wherever possible.

---

info@caldera.com
Copyright © 1994, 1997 Caldera, Inc. All rightsreserved.

Chapter 2 DPMS Function Reference

Introduction to DPMS Functions

This chapter describes all the DPMS function calls. All DPMS functions are supported by all servers except those functions which are explicitly optional, such as DPMS_ENABLE_DISABLE (see page 2-12), and those functions which can only be supported on a 386* or later CPU, such as the 32-bit interface and mapping functions. Servers which report a 386 or later CPU, that is, the value of the CPU field in the server structure is three or more, must support the 32-bit interface.

Generally, all functions preserve all registers unless they are used to return values. If AX is not used to return a value, then it is not set to zero or modified at all if the function is successful. If any 16-bit register can be modified, then the whole 32-bit register can also be modified even if it is not used to return a value. For example, the high word of EAX may be modified even if only AX is used for the return value.

The presence detection function may modify most registers as it may return values in currently unused registers in future versions, and so clients must not expect all registers to be preserved across this call. Refer to the description of the function DPMS_PRESENCE on page 2-6 to see which registers may be modified. The server does not enable interrupts while servicing function calls. It cannot stop third-party software that is used for mode transitions from enabling interrupts, but this should not occur.

Selectors passed by clients as parameters should correspond to writable data segments.

The 32-bit interface takes parameters and returns results in 32-bit registers for, for example, offsets and counts. For example, ES:EDI and ECX would be used in place of ES:DI and CX. This behavior is as expected and is similar to that of DPMI. There are two special cases.

· The function DPMS_D_ALLOC (see page 2-39) always takes CX and never ECX as the number of descriptors required, because the number can never be more than 8192.

· The inter-mode call functions always take CX and never ECX as the number of words to be copied from stack to stack, because real or virtual-8086 mode stacks are limited to 32768 words.

If 16 or 32-bit registers are used depending on whether the interface is 16 or 32-bit, then throughout this chapter, the "E" of the 32-bit register is enclosed in parentheses, for example, (E)CX.

Clients should be able to cope cleanly with error codes that are not currently defined.

Unloading DPMS Clients

TSRs are usually unloaded by running a second copy of the program with an appropriate command line switch. This second copy can also release DPMS resources. The server identifies clients from their API code and therefore, the second copy of the program must use the same API code as the original. This is easily done by copying the API code from the original program to the second copy, as if the second copy had registered as a client itself.

32-bit Return Values

32-bit values that are returned in two 16-bit registers are also returned in 32-bit registers where possible. If the server reports that the CPU is a 386 or later, then any register which returns the low word of a 32-bit value also returns the high word. For example, if a 32-bit value is returned in BX and CX (high word in BX, low word in CX), then the high word is returned in the high word of ECX as well as BX, that is, the ECX contains the entire 32-bit value, so a client with 386-specific code can choose to use BX and CX, or ECX.

Summary of DPMS Functions

This section summarizes the DPMS calls by function. The include file DPMS.INC lists the following function names and numbers:

INT 02Fh functions		
DPMS_PRESENCE	043E0h	Verifies the presence of a DPMS server.
DPMS_REGISTER	043E1h	Registers a DPMS client.
DPMS_ENABLE_DISABLE	043E2h	Disables or enables DPMS.
DPMS_STARTUP	043E3h	Issues a DPMS startup broadcast.
DPMS_EXIT	043E4h	Issues a DPMS exit broadcast.

General function		
DPMS_DEREGISTER	0000h	Deregister from the DPMS server as a DPMS client.

Control transfer to and from protected mode		
DPMS_CALL_PROT	0100h	Calls protected mode.
DPMS_CALL_REAL_RETF	0101h	Calls real or virtual-8086 mode (RETF return).
DPMS_CALL_REAL_IRET	0102h	Calls real or virtual-8086 mode (IRET return).
DPMS_CALL_REAL_INT	0103h	Calls a real or virtual-8086 mode interrupt handler.
DPMS_REG_DEF_PROT	0104h	Registers a default protected mode procedure.
DPMS_REG_DEF_REAL_RETF_16	0105h	

		Registers a default real or virtual-8086 mode procedure to be called from 16-bit protected mode with a RETF stack frame.
DPMS_REG_DEF_REAL_IRET_16	0106h	Registers a default real or virtual-8086 mode procedure to be called from 16-bit protected mode with an IRET stack frame.
DPMS_REG_DEF_REAL_INT_16	0107h	Registers a default real or virtual-8086 mode interrupt handler to be called from 16-bit protected mode.
DPMS_REG_DEF_REAL_RETF_32	0108h	Registers a default real or virtual-8086 mode procedure to be called from 32-bit protected mode with a RETF stack frame.
DPMS_REG_DEF_REAL_IRET_32	0109h	Registers a default real or virtual-8086 mode procedure to be called from 32-bit protected mode with an IRET stack frame.
DPMS_REG_DEF_REAL_INT_32	010Ah	Registers a default real or virtual-8086 mode interrupt handler to be called from 32-bit protected mode.

Descriptor management functions		
DPMS_D_ALLOC	0200h	Allocates descriptor(s).
DPMS_D_FREE	0201h	Frees a descriptor.
DPMS_D_ALIAS	0202h	Creates an alias descriptor.
DPMS_D_ALIAS_REAL	0203h	Builds an alias to a real mode segment.
DPMS_D_SET_BASE	0204h	Sets a descriptor base.
DPMS_D_SET_LIMIT	0205h	Sets a descriptor limit.
DPMS_D_SET_TYPE	0206h	Sets a descriptor type/attributes.
DPMS_D_GET_BASE	0207h	Gets a descriptor base.

Linear memory functions		
DPMS_M_FREESIZE	0300h	Returns the size of the largest free block.
DPMS_M_ALLOC	0301h	Allocates a block of extended memory.
DPMS_M_FREE	0302h	Frees a block of extended memory.
DPMS_M_MAP	0303h	Maps as linear memory.
DPMS_M_UNMAP	0304h	Unmaps as linear memory.
DPMS_M_GET_PAGE_TABLE	0305h	Gets page table entries.
DPMS_M_SET_PAGE_TABLE	0306h	Sets page table entries.
DPMS_M_MAP_SIZE	0307h	Gets the size of the largest mappable block.

Miscellaneous function

DPMS_M_RELOC	0400h	Relocates a segment to extended memory.

The following sections describe each DPMS function call.

DPMS_PRESENCE (043E0h)

Function

This function checks for the presence of a DPMS server. It is called by issuing an INT 02Fh request.

Entry Parameters

Register	Value
AX	DPMS_PRESENCE
BX	0000h
CX	04450h (`DP')
DX	04D53h (`MS')

Returned Values

Register	Value
If the function is successful	
CF	Clear
AX	0000h
ES:DI	Pointer to the DPMS server structure
If the function is unsuccessful	
AX	Non-zero

If either CF is set, or AX is non-zero, then a DPMS server is not present.

Explanation

This function may modify (E)AX, (E)CX, (E)DX, (E)BP, (E)SI, (E)DI, ES, FS, and GS. If a DPMS server is present, then the function returns ES:DI pointing to the DPMS server structure, which is defined in DPMS.INC as follows:

DPMS_Server_ID	db "DPMS"	ID string

DPMS_Server_DPMS_Version	db ?, ?	Version of the DPMS specification (major, minor)
DPMS_Server_OEM_Name	db 8 dup (?)	The OEM name for the server (space padded)
DPMS_Server_OEM_Version	db ?, ?	The OEM server version (major, minor)
DPMS_Server_Flags	dw ?	Flags (see the following explanation)
DPMS_Server_CPU	db ?	The CPU type

DPMS_Server_Flags

The DPMS_Server_Flags field is a 16-bit flag word of which the following bits are defined. Bits 3 to 15 are reserved.

DPMS_SERVER_FAST_RESET	0001h	The fast processor reset is used (applies to 80286 processors only).
DPMS_SERVER_ENABLED	0002h	The global DPMS enable or disable flag which is set if DPMS is enabled, and clear if it is not.
DPMS_SERVER_MAPPED	0004h	Memory mapping is not necessarily one-to-one, and mapping functions are supported.

Normally programs have to reset the 80286 to switch from protected to real mode, which is relatively slow. Some PCs have hardware to speed up this process. The fast reset bit is set to inform you if such hardware is used by the DPMS server.

It is recommended that all servers provide a mechanism by which DPMS can be enabled or disabled by the user: by typing DPMS ON or DPMS OFF at the system prompt, for example. This might be helpful if the user experiences problems with some DPMS software.

When DPMS is disabled, the register function, DPMS_REGISTER, is disabled so that clients cannot register; all other functions remain available. The DPMS_SERVER_ENABLED flag bit reflects the current state of DPMS, whether enabled or disabled.

DPMS_Server_DPMS_Version

DPMS servers supporting this version of the specification return the version number 1.0.

DPMS_Server_CPU

The differences between the 80286 and later processors mean that any DPMS server must determine the processor type on which it is running. This information is given to clients in the DPMS_Server_CPU field. Note that the DPMS server does not necessarily distinguish between the 80386 and later processor types. If a client needs this information, it must obtain it from another source. Even so, clients should be aware that this field might be higher than three.

DPMS_SERVER_CPU_286	0002h	80286 processor

DPMS_SERVER_CPU_386	0003h	80386 or later processors

DPMS functions are passed a function number in the register AX. These numbers are defined in the include file DPMS.INC as equates to the function name that heads each function description in this chapter. For instance, to call DPMS_M_FREESIZE (see page 2-53) from real mode, use:

```
mov     ax, DPMS_M_FREESIZE
call    real_entry
```

You can call most DPMS functions from either real or protected mode, except DPMS_M_RELOC (see page 2-66), which you must call from real mode, and the real and protected mode call-up and call-down functions, which can be called only when in the appropriate mode.

DPMS_REGISTER (043E1h)

Register As A DPMS Client

Function

This function registers a new DPMS client, and is called by issuing an INT 02Fh.

Entry Parameters

Register	Value
AX	DPMS_REGISTER
CX	Default protected mode stack size. This is zero if no default protected mode stack is required.
ES:DI	Pointer to the DPMS client interface structure.

Returned Values

Register	Value
If the function is successful	
CF	Clear
AX	0000h
ES:DI	Unmodified, DPMS client interface structure filled in
If the function is unsuccessful	
CF	Set

AX	Error code (see "Error Codes" on page A-1), including DPMS_E_NOT_SUPP (DPMS is disabled) DPMS_E_RESOURCE_NA (Resource unavailable) DPMS_E_DESC_NA (Descriptor unavailable) DPMS_E_LIN_NA (Linear memory unavailable) DPMS_E_PHYS_NA (Physical memory unavailable)

Explanation

The DPMS client interface structure is defined as follows:

DPMS_IF_Version	dw 0	The structure version and flags.
DPMS_IF_Name	db 8 dup (?)	The client name (space padded).
DPMS_IF_Real_Entry	db 7 dup (?)	The real mode API code.
DPMS_IF_Real_Entry_Ret	db 0C3H	Space for a return instruction; the default is a near return.
DPMS_IF_Prot_Entry_16	db 7 dup (?)	Protected mode API code (16-bit).
DPMS_IF_Prot_Entry_16_Ret	db 0C3H	Space for a return instruction; the default is a near return.
DPMS_IF_Prot_Entry_32	db 9 dup (?)	Protected mode API code (32-bit); this is undefined if the DPMS_Server_CPU field in the server structure is less than three.
DPMS_IF_Prot_Entry_32_Ret	db 0C3H	Space for a return instruction; the default is a near return.

The version and flags (DPMS_IF_Version) and client name (DPMS_IF_Name) fields must be filled in by the client before making the call. The version and flags field is for upward compatibility; for the current version it must be zero. The name field is available to statistical and diagnostic programs to assist in identifying resource ownership, and must be padded with spaces.

If successful, CF and AX are clear and the API code fields in the client interface structure are filled in with code which, when executed, calls the DPMS API and also contains a client ID. The return instruction fields can be filled in by the client at any time, and are provided so that the client can call the API code without copying it out of the structure. The client can also move or copy the API code fields anywhere, for example to inline code, as long as all bytes in the field are copied. The three API code fields do not have to be kept together.

The default protected mode stack size is passed in CX. Refer to the description of the function DPMS_CALL_PROT on page 2-18 for a description of protected mode stacks.

When DPMS is disabled, this function is disabled so that clients cannot register.

All further DPMS functions are called using the API code in the client interface structure. This is an unusual interface but it has a number of advantages. If clients called the server using standard segment:offset entry points, either they would have to load a register with a client handle, or the server would have to allocate space for a separate entry point for each possible client. The use of the API code is consistent with the interface used to call default procedures. The API code often consists of a far call to the server but can contain other code, such as an interrupt instruction or other trap, which can be more efficient.

DPMS_ENABLE_DISABLE (043E2h)

Enable or Disable DPMS

Function

This function enables or disables DPMS. It is called via INT 02Fh.

Entry Parameters

Register	Value
AX	DPMS_ENABLE_DISABLE
BX	0 to disable DPMS, 1 to enable it

Returned Values

Register	Value
If the function is successful	
CF	Clear
AX	0

Explanation

This function is optional and should normally only be used by system software. When DPMS is disabled, the register function is disabled so that clients cannot register, although all other functions work normally.

DPMS_STARTUP (043E3h)

DPMS Server Startup Broadcast

Function

On startup but before installation, a potential DPMS server issues an INT 02Fh, as follows:

Entry Parameters

Register	Value
AX	DPMS_STARTUP
BX	0
CX	04450h (`DP')
DX	04D53h (`MS')
ES:DI	Pointer to the DPMS server structure

Returned Values

Register	Value
If it is alright for the server to load	
BX	Unmodified
If it is not alright for the server to load	
BX	Bit 0 set

Explanation

A potential server issues the startup broadcast when running as a normal DOS program, so resident programs intercepting the broadcast can make normal DOS and BIOS calls without considering re-entrancy. Refer to the explanation of the function DPMS_PRESENCE on page 2-6 for a description of the DPMS server structure.

The startup broadcast has three main uses.

· Potential clients that are already resident can use the broadcast to output a message informing the user that they are DPMS aware, and usually must be loaded after DPMS.

· Resident software can modify its behavior while a DPMS server is present, if required.

· Resident software might be incompatible with DPMS, and can force the potential server not to install.

Potential DPMS servers always search for an existing DPMS server before installing, therefore existing servers need not listen for the startup broadcast in order to prevent other servers from loading.

DPMS_EXIT (043E4h)

DPMS Server Exit Broadcast

Function

Before unloading, a DPMS server issues an INT 02Fh, as follows:

Entry Parameters

Register	Value
AX	DPMS_EXIT
BX	0
CX	04450h (`DP')
DX	04D53h (`MS')
ES:DI	Pointer to the DPMS server structure

Returned Values

Register	Value
If it is alright for the server to unload	
BX	Unmodified
If it is not alright for the server to unload	
BX	Bit 0 set

Explanation

Servers never unload while any client is registered. Refer to the explanation of the function DPMS_PRESENCE on page 2-6 for a description of the DPMS server structure.

DPMS_DEREGISTER (0000h)

Deregister As A DPMS Client

Function

This function complements the register function DPMS_REGISTER (see page 2-9), and must be called by any client that no longer needs any DPMS resources. The function enables the server to free any resources that were allocated to that client when it registered. It is called via the real mode API code.

Entry Parameters

Register	Value
AX	DPMS_DEREGISTER

Returned Values

Register	Value
If the function is successful	
CF	Clear
AX	Unmodified
If the function is unsuccessful	
CF	Set
AX	Error code (see "Error Codes" on page A-1), including DPMS_E_UNKNOWN_CLIENT (Unknown client)

Explanation

This call does not free any descriptors, memory, and so forth, allocated by the client via the DPMS API. These must be freed explicitly using the appropriate DPMS calls before making the deregister call. Once deregistered, the API code copied to the client interface structure by the server when the client registered must not be used. No further DPMS calls must be made by the client after deregistering unless the client registers with the server again and gets new API code.

DPMS_CALL_PROT (0100h)

Protected Mode Call-up

Function

This function calls a protected mode procedure. It is called via real mode API code.

Entry Parameters

Register	Value
AX	DPMS_CALL_PROT
CX	Number of words to copy from real mode to protected mode stack
ES:DI	Pointer to the register structure (see Figure 2-1)

Returned Values

Register	Value
If the function is successful	
CF	Clear
AX	Unmodified
If the function is unsuccessful	
CF	Set
AX	Error code (see "Error Codes" on page A-1), including, DPMS_E_PROTECTED (Cannot switch to protected mode) DPMS_E_RESOURCE_NA (Resource unavailable)

The register structure values are updated.

Explanation

The register structure is a parameter block in real mode memory. Its structure is shown in Figure 2-1.

The address of the protected mode procedure to be called, and the values to be loaded into the registers on entry to that procedure, are passed to the DPMS server in a register structure. On return from the protected mode procedure, the (E)AX, (E)BX, (E)CX, (E)DX, (E)BP, (E)SI, (E)DI, DS, ES, FS, GS, and (E)FLAGS registers are copied back into the register structure. A far call is made to the procedure, and therefore it must execute a far return. Any words copied to the protected mode stack from the real mode stack appear on the stack above the far return address. The calling procedure must remove any parameters passed on the stack, and therefore, the protected mode procedure should not clean up the stack on exit because the DPMS server does this. However, any parameters placed on the real mode stack before calling the DPMS_CALL_PROT function (see page 2-18) are still there on return from the server and the client must clean them up.

Make sure all selector values are valid. DPMS has functions you can call while in real mode to create valid selectors. Segment registers that are unused must be set to zero.

There is a reserved field in the register structure used for inter-mode calls which is there to make the structure compatible with the PUSHAD instruction. The field is occupied by (E)SP which is defined in DPMS.INC as DPMS_Reg_ESP or DPMS_Reg_SP, is reserved but may be set to any value by the client before making the inter-mode call, and may be modified by the server during the call. Normally the server sets this field to the (E)SP returned from the destination procedure, but this does not happen in all cases.

If the server reports that the CPU is a 386 or later, then the high words of the EIP, ESP, and EFLAGS fields in the register structure must always be defined by the client, even if the corresponding CS or SS segments are 16-bit. The high word of the EFLAGS field is used by 386 servers, but some bits are set or cleared as appropriate by the server, particularly the VM, NT, and IOPL bits.

Figure 2-1
Register Structure

Offset	**Register**
000h	EDI
004h	ESI
008h	EBP
00Ch	Reserved, may be used by DPMS server.
010h	EBX
014h	EDX
018h	ECX
01Ch	EAX
020h	EIP
024h	CS
028h	EFLAGS
02Ch	ESP
030h	SS
034h	ES
038h	DS
03Ch	FS
040h	GS

DPMS.INC provides the following definitions for the register structure that allows you to declare and initialize your parameter blocks and to refer to all register fields symbolically.

DPMS_Reg_Struc	struc	
DPMS_EDI	dd ?	
DPMS_ESI	dd ?	
DPMS_EBP	dd ? dw 0, 0	Reserved, may be used by the DPMS server
DPMS_EBX	dd ?	
DPMS_EDX	dd ?	

DPMS_ECX	dd ?	
DPMS_EAX	dd ?	
DPMS_EIP	dd ?	
DPMS_CS	dw ? db 0, 0	
DPMS_EFLAGS	dd ?	
DPMS_ESP	dd ?	
DPMS_SS	dw ? db 0, 0	
DPMS_ES	dw ? db 0, 0	
DPMS_DS	dw ? db 0, 0	
DPMS_FS	dw ? db 0, 0	
DPMS_GS	dw ? db 0, 0	
DPMS_Reg_Struc	ends	

DPMS.INC also provides definitions for the 8-bit and 16-bit register within it symbolically. These definitions include, for example, DPMS_AL, DPMS_AH, and DPMS_AX.

Protected Mode Stacks

Real or virtual-8086 and protected mode stacks must be separate and clients cannot assume anything about the amount of stack used by the server. This means that there must be no possibility of one corrupting the other and so they should be in separate areas of memory. However, they may share common segment base addresses and therefore use common pointers within those segments. For example, (E)BP can be used to point to parameters on a real or virtual-8086 mode stack which can be accessed using the SS of the protected mode stack with the same (E)BP.

· Hardware or software interrupts that occur while protected mode code executes are reflected to real mode. The DPMS server uses the real mode stack of the last client that called up to protected mode while reflecting the interrupt. Therefore, clients' protected mode functions should not put extra data on the real mode caller's stack at a memory address immediately preceding the top of the stack at call-up time. For the same reason, real mode clients must not use a protected mode stack descriptor that is an alias of the real mode stack segment while using the same stack pointer value.

· If a stack selector of zero is passed to the server, then the server provides a default protected mode stack. Each stack is at least as big as the default size requested by the client when it registered.

· A protected mode default stack may be 16 or 32-bit, but if the client's protected mode code is 16-bit, then the default stack is guaranteed to be within the first 64 KB of the stack segment. Therefore, 16-bit clients can use 16-bit pointers to access parameters on the stack. However, 32-bit clients must always use 32-bit pointers to access parameters on the stack.

· Clients have the choice of specifying SS:(E)SP, or using a default stack provided by the DPMS server. Specifying SS:(E)SP has the advantage that the client can use SS to reference variables, but the disadvantage is that the client must either prevent re-entrancy, or take care of the complications of re-entrancy.

· The value of (E)SP on entry to protected mode is different from the supplied value because the DPMS server pushes a return address and possibly other data on the protected mode stack before passing control to the protected mode procedure.

· Call-ups and call-downs (including reflected interrupts) must nest strictly. You cannot return from a call-up or call-down if you have not yet returned from all more recent call-downs or call-ups. In a multitasking system calls may not be strictly nested across the whole system, but must be strictly nested within each virtual machine.

DPMS_CALL_REAL_RETF (0101h)

Real Mode Call-down

Function

This function calls a real mode procedure with a RETF stack frame. It is called via protected mode API code.

Entry Parameters

Register	Value
AX	DPMS_CALL_REAL_RETF
CX	Number of words to copy from protected mode to real mode stack
ES:(E)DI	Pointer to a register structure

Returned Values

Register	Value
If the function is successful	
CF	Clear
AX	Unmodified

If the function is unsuccessful	
CF	Set
AX	Error code (see "Error Codes" on page A-1), including DPMS_E_RESOURCE_NA (Resource unavailable)

The values in the register structure are updated.

The register structure has the same layout as the one used for calling from real mode to protected mode in Figure 2-1. However, the interpretation of some fields is different, as follows:

· All segment values in the register structure are real mode paragraphs, not protected mode selectors.

· Only the low order 16 bits of EIP are used.

· If a stack segment of zero is passed to the server, then the server provides a default real mode stack. The most recent protected mode call-up that has not returned is used for loading SS:SP. This makes stack usage more consistent with a real mode environment.

· The value of SP on entry to the real mode code is different from the supplied value, because the DPMS server pushes a return address, and possibly other data, on the real mode stack before passing control to the real mode code.

Refer to the description of the function DPMS_CALL_PROT on page 2-18 for more information.

DPMS_CALL_REAL_IRET (0102h)

Real Mode Call-down

Function

This function calls a real mode procedure with an IRET stack frame. It is called via protected mode API code.

Entry Parameters

Register	Value
AX	DPMS_CALL_REAL_IRET
CX	Number of words to copy from protected mode to real mode stack
ES:(E)DI	Pointer to a register structure

Returned Values

Register	Value
If the function is successful	
CF	Clear
AX	Unmodified
If the function is unsuccessful	
CF	Set
AX	Error code (see "Error Codes" on page A-1), including DPMS_E_RESOURCE_NA (Resource unavailable)

The values in the register structure are updated.

Explanation

Refer to the explanation of DPMS_CALL_REAL_RETF on page 2-24 for a more detailed description.

DPMS_CALL_REAL_INT (0103h)

Real Mode Call-down

Function

This function calls a real mode interrupt routine with an IRET stack frame. It is called via protected mode API code.

Entry Parameters

Register	Value
AX	DPMS_CALL_REAL_INT
CX	Number of words to copy from protected mode to real mode stack
BL	Interrupt number
ES:(E)DI	Pointer to a register structure

Returned Values

Register	Value
If the function is successful	
CF	Clear
AX	Unmodified

If the function is unsuccessful	
CF	Set
AX	Error code (see "Error Codes" on page A-1), including DPMS_E_RESOURCE_NA (Resource unavailable)

The values in the register structure are updated.

Explanation

DPMS_CALL_REAL_INT operates similarly to DPMS_CALL_REAL_IRET (see page 2-26), except that it ignores the CS:IP values in the register structure, and instead uses the real mode interrupt vector whose number is passed in BL.

Because of the overheads involved in building and executing a real mode INT instruction, the DPMS server may emulate the instruction by pushing the FLAGS and calling the interrupt service routine, rather than executing an actual INT.

The CS and EIP fields of the register structure used for inter-mode calls are preserved by the server, except when calling down from protected mode to a real or virtual-8086 mode interrupt handler by the interrupt number, DPMS_CALL_REAL_INT, in which case the server may modify the CS and EIP fields.

DPMS_REG_DEF_PROT (0104h)

Register Default Protected Mode Procedure

Function

This function registers a default protected mode procedure. It is called via the real or protected mode API code.

Entry Parameters

Register	Value
AX	DPMS_REG_DEF_PROT
ES:(E)DI	Pointer to the DPMS default register structure

Returned Values

Register	Value
If the function is successful	

CF	Clear
AX	Unmodified
If the function is unsuccessful	
CF	Set
AX	Error code (see "Error Codes" on page A-1), including DPMS_E_BAD_SEL (Invalid selector)

Explanation

Most calls from real or virtual-8086 to protected mode, and from protected to real or virtual-8086 mode, are to addresses determined at initialization time. Many calls require parameters to be passed in registers that have to be stored in a DPMS register structure, if calling via the call-up or call-down functions DPMS_CALL_PROT (see page 2-18), DPMS_CALL_REAL_IRET (see page 2-26), DPMS_CALL_REAL_RETF (see page 2-24), and DPMS_CALL_REAL_INT (see page 2-27). The following is an alternative way of making inter-mode calls, in addition to the functions mentioned previously.

Using the alternative method, the client registers a default procedure to be called, together with default values for certain registers, including the segment registers. This is done by passing the address of a default register structure to the server. The server fills in part of the structure with the code necessary to make the inter-mode call. This code also contains a client ID. The client can register almost any number of default procedures in this way; each procedure is called via the code in the corresponding default register structure.

All registers, other than those defined in the default register structure, including the flags, are normally passed to and returned from the destination procedure. However, it might be the case that the server cannot make the call, for instance if some other program has gained control and the server cannot get into protected mode. In this case the server sets the carry flag, and sets the error field in the default register structure to one. If the call is successful, the server does not modify the carry flag, and for speed, does not set the error field to zero, so it is the responsibility of any client that uses this field to ensure that it is set to zero before making the call. In practice, this is most efficiently done by initializing the field to zero and only resetting it to zero if an error occurs.

The default register structure is defined as follows:

DPMS_Def_EIP	dd ?	
DPMS_Def_CS	dw ? db 0, 0	
DPMS_Def_Words	dw ?	Words to be copied from stack to stack
DPMS_Def_Error	db 0	Set to non-zero on return if the call could not be made, zeroed by client
DPMS_Def_Res	db ?	Reserved, may be used by the DPMS server

DPMS_Def_ESP	dd ?	
DPMS_Def_SS	dw ?	
DPMS_Def_ES	dw ? db 0, 0	
DPMS_Def_DS	dw ? db 0, 0	
DPMS_Def_FS	dw ? db 0, 0	
DPMS_Def_GS	dw ? db 0, 0	
DPMS_Def_Entry	9 dup (?)	API code filled in by the DPMS server

Unlike the client interface structure, the API code in the default register structure is not followed by a defined return instruction field. This is because the API code can be followed by any code that the client wishes to execute after the call, and if the return instruction field was defined, then that would make the structure less useful. In general, the API code must be followed by return code defined by the client.

All fields in the default register structure, except the API code, must be filled in by the client before registering the procedure, and normally, only the error field will be altered by the client after the procedure is registered. Other fields may be changed after registering the call, but there are some restrictions due to the fact that the API code may depend upon the types of fields.

The entry point fields may only be changed if the destination mode and type remains unchanged, for example, 16-bit protected mode. The segment register fields may not be changed from a special value, for example, zero, to a normal value, or from a normal value to a special one. The word count field may not be changed from zero to non-zero, or non-zero to zero. Breaking these rules after registering the call might work in practice but cannot be relied upon under all servers, because the API code used varies with the type of call and other parameters. However, the entire structure may be moved in memory after registering the default procedure.

The server does not modify any fields in the default register structure when making calls, except the error and reserved fields, and the CS and EIP fields if calling down from protected mode to a real or virtual-8086 interrupt handler by interrupt number.

DPMS defines some special values for the DS, ES, FS and GS segment registers when calling from real or virtual-8086 to protected mode, or from protected mode to real or virtual-8086 mode. Instead of loading the segment register with the specified value, the server interprets the value as follows:

Name (in DPMS.INC)	Value	Interpretation

DPMS_SEG_DEFAULT	0	The segment register is loaded with its default value, or zero if no default is defined.
DPMS_SEG_ALIAS	1	The segment register is loaded with a value that corresponds to its value in the opposite mode if possible, so call-ups create a descriptor corresponding to the real or virtual-8086 mode segment, and load the segment register with the selector for that descriptor. Call downs get the base address of the descriptor for the protected mode selector and convert that to a real or virtual-8086 mode segment if possible.
Other		The value specified is loaded into the segment register.

If speed is important, the client should not load DPMS_SEG_ALIAS if the segment is always going to be the same.

Refer to the description of the function DPMS_CALL_PROT on page 2-18, because the information about that function also applies to the use of default call-up procedures.

DPMS_REG_DEF_REAL_RETF_16 (0105h)

Register Default Real or Virtual-8086 Mode Procedure

Function

This function registers a default real or virtual-8086 mode procedure when the procedure is to be called from 16-bit protected mode with a RETF stack frame. It is called via real or protected mode API code.

Entry Parameters

Register	Value
AX	DPMS_REG_DEF_REAL_RETF_16
ES:(E)DI	Pointer to the DPMS default register structure

Returned Values

Register	Value
If the function is successful	
CF	Clear
AX	Unmodified
If the function is unsuccessful	
CF	Set
AX	Error code (see "Error Codes" on page A-1)

Explanation

Refer to the description of the function DPMS_REG_DEF_PROT on page 2-29 for more information about default call-up and call-down procedures.

DPMS_REG_DEF_REAL_IRET_16 (0106h)

Register Default Real or Virtual-8086 Mode Procedure

Function

This function registers a default real or virtual-8086 mode procedure when the procedure is to be called from 16-bit protected mode with an IRET stack frame. It is called via real or protected mode API code.

Entry Parameters

Register	Value
AX	DPMS_REG_DEF_REAL_IRET_16
ES:(E)DI	Pointer to the DPMS default register structure

Returned Values

Register	Value
If the function is successful	
CF	Clear
AX	Unmodified
If the function is unsuccessful	
CF	Set
AX	Error code (see "Error Codes" on page A-1)

Explanation

Refer to the description of the function DPMS_REG_DEF_PROT on page 2-29 for more information about default call-up and call-down procedures.

DPMS_REG_DEF_REAL_INT_16 (0107h)

Register Default Real or Virtual-8086 Mode Procedure

Function

This function registers a default real or virtual mode 8086 procedure when the interrupt handler is to be called from 16-bit protected mode. It is called via real or protected mode API code.

Entry Parameters

Register	Value
AX	DPMS_REG_DEF_REAL_INT_16
BL	Interrupt number
ES:(E)DI	Pointer to the DPMS default register structure

Returned Values

Register	Value
If the function is successful	
CF	Clear
AX	Unmodified
If the function is unsuccessful	
CF	Set
AX	Error code (see "Error Codes" on page A-1)

Explanation

Refer to the description of the function DPMS_REG_DEF_PROT on page 2-29 for more information about default call-up and call-down procedures.

DPMS_REG_DEF_REAL_RETF_32 (0108h)

Register Default Real or Virtual-8086 Mode Procedure

Function

This function registers a default real or virtual-8086 mode procedure when the procedure is to be called from 32-bit protected mode with a RETF stack frame. It is called via real or protected mode API code.

Entry Parameters

Register	Value
AX	DPMS_REG_DEF_REAL_RETF_32
ES:(E)DI	Pointer to the DPMS default register structure

Returned Values

Register	Value
If the function is successful	
CF	Clear
AX	Unmodified
If the function is unsuccessful	
CF	Set
AX	Error code (see "Error Codes" on page A-1)

Explanation

Refer to the description of the function DPMS_REG_DEF_PROT on page 2-29 for more information about default call-up and call-down procedures.

DPMS_REG_DEF_REAL_IRET_32 (0109h)

Register Default Real or Virtual-8086 Mode Procedure

Function

This function registers a default real or virtual-8086 mode procedure when the procedure is to be called from 32-bit protected mode with an IRET stack frame. It is called via real or protected mode API code.

Entry Parameters

Register	Value
AX	DPMS_REG_DEF_REAL_IRET_32
ES:(E)DI	Pointer to the DPMS default register structure

Returned Values

Register	Value
If the function is successful	
CF	Clear
AX	Unmodified
If the function is unsuccessful	
CF	Set
AX	Error code (see "Error Codes" on page A-1)

Explanation

Refer to the description of the function DPMS_REG_DEF_PROT on page 2-29 for more information about default call-up and call-down procedures.

DPMS_REG_DEF_REAL_INT_32 (010Ah)

Register Default Real or Virtual-8086 Mode Procedure

Function

This function registers a default real or virtual-8086 mode procedure when the interrupt handler is to be called from 32-bit protected mode. It is called via real or protected mode API code.

Entry Parameters

Register	Value
AX	DPMS_REG_DEF_REAL_INT_32
BL	Interrupt number
ES:(E)DI	Pointer to the DPMS default register structure

Returned Values

Register	Value
If the function is successful	
CF	Clear
AX	Unmodified

If the function is unsuccessful	
CF	Set
AX	Error code (see "Error Codes" on page A-1)

Explanation

Refer to the description of the function DPMS_REG_DEF_PROT on page 2-29 for more information about default call-up and call-down procedures.

DPMS_D_ALLOC (0200h)

Allocate Descriptor

Function

This function allocates one or more contiguous protected mode descriptors. It is called via protected or real mode API code.

Entry Parameters

Register	Value
AX	DPMS_D_ALLOC
CX	Number of descriptors required

Returned Values

Register	Value
If the function is successful	
CF	Clear
AX	Selector for the (first) new descriptor
If the function is unsuccessful	
CF	Set
AX	Error code (see "Error Codes" on page A-1,) including DPMS_E_DESC_NA (Descriptor unavailable)

Explanation

This call allocates new descriptors and initializes each to be a data segment descriptor with a limit of 64 KB. DPMS sets the privilege level; the client cannot modify it. The descriptors can be in either the GDT or the LDT.

The limit and attributes of newly-allocated descriptors are set to the default of 64 KB data, but the base is undefined. Clients can load selectors for newly-allocated descriptors but should not access anything using these descriptors until the base has been set. The server can then initialize the base so that the descriptor references memory that is not present, or take other steps to guard against this.

If the client requests more than one descriptor, contiguous descriptors are allocated. As the index for the descriptor consists of the highest 13 bits in a selector, you obtain selector values for subsequent descriptors by adding multiples of eight to the returned value.

You can use the DPMS_D_SET_BASE function (see page 2-47) to point this descriptor to a linear memory block in extended memory allocated via DPMS_M_ALLOC (see page 2-54), or you can use DPMS_D_ALIAS_REAL (see page 2-45) to turn it into a protected mode alias for any real mode address.

Remember to call DPMS_D_FREE (see page 2-41) to free all descriptors that are no longer required, because descriptors are a limited resource.

DPMS_D_FREE (0201h)

Free Descriptor

Function

This function frees the descriptors allocated by DPMS_D_ALLOC (see page 2-39), DPMS_D_ALIAS (see page 2-43), and DPMS_M_RELOC (see page 2-66). It is called via protected or real mode API code.

Entry Parameters

Register	Value
AX	DPMS_D_FREE
BX	Selector to free

Returned Values

Register	Value
If the function is successful	
CF	Clear

AX	Unmodified
If the function is successful	
CF	Set
AX	Error code (see "Error Codes" on page A-1) including DPMS_E_BAD_SEL (Invalid selector)

Explanation

After this call, do not use the selector, because an exception is raised if it is loaded into a segment register.

The linear memory corresponding to the descriptor is *not* freed by this call. You must call DPMS_M_FREE (see page 2-56) to free it before freeing the descriptor.

Each call to DPMS_D_FREE frees only one descriptor. If you allocated several contiguous descriptors with DPMS_D_ALLOC (see page 2-39), you must call DPMS_D_FREE for each of them.

DPMS_D_ALIAS (0202h)

Allocate Alias Descriptor

Function

This function allocates a descriptor that is an alias to an existing descriptor to a segment. It is called via protected or real mode API code.

Entry Parameters

Register	Value
AX	DPMS_D_ALIAS
BX	Selector for the existing descriptor

Returned Values

Register	Value
If the function is successful	
CF	Clear
AX	Selector for the alias descriptor
If the function is unsuccessful	
CF	Set

AX	Error code (see "Error Codes" on page A-1) including DPMS_E_DESC_NA (Descriptor unavailable) DPMS_E_BAD_SEL (Invalid selector)

Explanation

The new descriptor describes a data segment that otherwise matches the limit and base of the existing descriptor. You can use this feature to get write access to a code segment. Instead of writing via the code segment's selector, which is illegal in protected mode, simply write via the alias descriptor instead.

DPMS_D_ALIAS_REAL (0203h)

Build Alias to Real Mode Segment

Function

This function builds a descriptor that is an alias to a real or virtual-8086 mode segment. It is called via real or protected mode API code.

Entry Parameters

Register	Value
AX	DPMS_D_ALIAS_REAL
BX	Selector for the descriptor
CX	Real mode segment (paragraph) value

Returned Values

Register	Value
If the function is successful	
CF	Clear
AX	Unmodified
If the function is unsuccessful	
CF	Set
AX	Error code (see "Error Codes" on page A-1) including DPMS_E_BAD_SEL (Invalid selector)

Explanation

This function converts the descriptor into a data segment with a limit of 0FFFFH and a base of 16* (real mode segment).

Use this call for high frequency address translation, because it does not have to allocate a new descriptor each time. Also there is no risk of running out of descriptors at run time.

In performance critical sections of code, call this function from protected mode, because it is significantly faster than from real mode.

DPMS_D_SET_BASE (0204h)

Set Descriptor Base Address

Function

This function sets the base of a descriptor to a given linear address value. It is called via protected or real mode API code.

Entry Parameters

Register	Value
AX	DPMS_D_SET_BASE
BX	Selector for the descriptor
CX,DX	Linear address

Returned Values

Register	Value
If the function is successful	
CF	Clear
AX	Unmodified
If the function is unsuccessful	
CF	Set
AX	Error code (see "Error Codes" on page A-1) including DPMS_E_BAD_SEL (Invalid selector) DPMS_E_BAD_LIN_ADDR (Invalid linear memory address)

Explanation

The linear address is 32 bits. The high word is passed in CX, the low word in DX.

DPMS_D_SET_LIMIT (0205h)

Set Descriptor Limit

Function

This function sets the limit of a descriptor. It is called via real or protected mode API code.

Entry Parameters

Register	Value
AX	DPMS_D_SET_LIMIT
BX	Selector for the descriptor
CX,DX	New limit (size-1)

Returned Values

Register	Value
If the function is successful	
CF	Clear
AX	Unmodified
If the function is unsuccessful	
CF	Set
AX	Error code (see "Error Codes" on page A-1) including DPMS_E_BAD_VALUE (Invalid value) DPMS_E_BAD_SEL (Invalid selector)

Explanation

The limit is the number of bytes minus one. The granularity of the descriptor is automatically set as required by the DPMS server depending on the limit required. Clients running on an 80286 are limited to 64 KB segments, so do not set the limit to greater than 0FFFFh.

DPMS_D_SET_TYPE (0206h)

Set Descriptor Type or Attributes

Function

This function sets a descriptor's type and attributes. It is called via real or protected mode API code.

Entry Parameters

Register	Value
AX	DPMS_D_SET_TYPE
BX	Selector for the descriptor
CL	Type or attributes to be set (byte 5 of the descriptor)
CH	386 extended type or attributes to be set (byte 6 of the descriptor)

Returned Values

Register	Value
If the function is successful	
CF	Clear
AX	Unmodified
If the function is unsuccessful	
CF	Set
AX	Error code (see "Error Codes" on page A-1) including DPMS_E_BAD_VALUE (Invalid value) DPMS_E_BAD_SEL (Invalid selector)

Explanation

Refer to the Intel* 80286 and 386 programmer's reference documentation for a description of the type and attribute fields in descriptors.

The parameter passed in CL is used to set the byte at offset 5 within the descriptor, that is, the byte containing P, DPL, DT and Type. The DPL specified in register CL must be 0. Do not make assumptions about the actual privilege level that DPMS uses.

DPMS uses the parameter passed in CH to set bits 5 and 6 of the byte at offset 6 within the descriptor. The AVL bit is reserved for use by the DPMS server, and should be set to zero in CH. The granularity bit is set or cleared when the limit is set.

DPMS_D_GET_BASE (0207h)

Get Descriptor Base

Function

This function gets the base of a descriptor. It is called via real or protected mode API code.

Entry Parameters

Register	Value
AX	DPMS_D_GET_BASE
BX	Selector for the descriptor

Returned Values

Register	Value
If the function is successful	
CF	Clear
CX,DX	Linear address of the base of the segment
AX	Unmodified
If function is unsuccessful	
CF	Set
AX	Error code (see "Error Codes" on page A-1) including DPMS_E_BAD_SEL (invalid selector)

Explanation

The returned value is 32 bits. The high word is returned in CX, the low word in DX.

DPMS_M_FREESIZE (0300h)

Get Size of Largest Free Block

Function

This function gets the size of the largest free block in linear memory. It is called via real or protected mode API code.

Entry Parameters

Register	Value
AX	DPMS_M_FREESIZE

Returned Values

Register	Value
If the function is successful	
CF	Clear
AX	Unmodified
BX,CX	Number of bytes in the largest free block
If the function is unsuccessful	
CF	Set
AX	Error code (see "Error Codes" on page A-1) including DPMS_E_LIN_NA (Linear memory unavailable) DPMS_E_PHYS_NA (Physical memory unavailable)

Explanation

The returned value is 32 bits. The high word is returned in BX, the low word in CX.

This function returns the amount of memory that is available to clients via DPMS_M_ALLOC (see page 2-54).

DPMS_M_ALLOC (0301h)

Allocate Linear Memory

Function

This function allocates a block of linear memory. It is called via real or protected mode API code.

Entry Parameters

Register	Value
AX	DPMS_M_ALLOC
BX,CX	Number of bytes required

Returned Values

Register	Value
If the function is successful	
CF	Clear

BX,CX	Linear base address of the allocated block
SI,DI	Handle of the allocated block
AX	Unmodified
If the function is unsuccessful	
CF	Clear
AX	Error code (see "Error Codes" on page A-1) including DPMS_E_RESOURCE_NA (Resource unavailable) DPMS_E_LIN_NA (Linear memory unavailable) DPMS_E_PHYS_NA (Physical memory unavailable)

Explanation

The returned values are 32 bits. The high order words are returned in BX and SI, and the low words are returned in CX and DI.

This call allocates physical memory and maps it into DPMS linear address space. The granularity of memory allocation is usually larger than a single byte, and is often 4 KB. Therefore, multiple allocations of a few bytes may use 4 KB each.

You may use one or more calls to DPMS_D_SET_BASE (see page 2-47) to map descriptors to anywhere within this memory block in order to access memory.

Memory allocated using this method is obtained directly from the underlying memory manager. No method exists to allow this memory to be shared with other applications, for example, MS Windows. Allocate memory directly from the underlying memory manager, if you want to share this memory with normal DOS programs.

DPMS_M_FREE (0302h)

Free Linear Memory

Function

This function frees a linear memory block. It is called via real or protected mode API code.

Entry Parameters

Register	Value
AX	DPMS_M_FREE
SI,DI	Handle of the block

Returned Values

Register	Value
If the function is successful	
CF	Clear
AX	Unmodified
If the function is unsuccessful	
CF	Set
AX	Error code (see "Error Codes" on page A-1) including DPMS_E_BAD_HANDLE (Invalid handle)

Explanation

This call frees physical memory allocated via DPMS_M_ALLOC (see page 2-54) or DPMS_M_RELOC (see page 2-66). If you try to access linear addresses within that block after it has been freed, a page fault may result. This call does not free any descriptors that might still refer to this memory block. Do not reference memory pointed to by such descriptors after this function call.

DPMS_M_MAP (0303h)

Map as Linear Memory

Function

This optional function maps physical memory into contiguous linear DPMS address space. This function is called via real or protected mode API code.

Entry Parameters

Register	Value
AX	DPMS_M_MAP
ES:(E)DI	Pointer to Virtual DMA Services (VDS) DMA Descriptor Structure (DDS)

Returned Values

Register	Value
If the function is successful	
CF	Clear
BX,CX	Linear address
SI,DI	Handle

AX	Unmodified
If the function is unsuccessful	
CF	Set
AX	Error code (see "Error Codes" on page A-1) including DPMS_E_NOT_SUPP (Unsupported function) DPMS_E_RESOURCE_NA (Resource unavailable) DPMS_E_LIN_NA (Linear memory unavailable)

Explanation

This function is useful for making memory allocated via XMS, or memory-mapped devices, or physical memory returned via the VDS, addressable for protected mode DPMS clients. On some implementations, 80286 for example, this function is not supported. Ignore the error code "Unsupported function" that is returned in these cases, because the physical and linear addresses are identical.

This function takes physical addresses as input and maps the physical blocks into contiguous DPMS linear address space. If the client has allocated the physical memory via some interface other than DPMS it should ensure that it has the physical addresses of that memory. One way to obtain the physical addresses is via Virtual DMA Services (VDS), which is an interface designed for software such as the BIOS which must program DMA controllers and therefore must convert linear addresses into physical ones. It is beyond the scope of this document to describe VDS, but a summary of the recommended VDS function is provided below.

The number, addresses, and sizes of the physical memory blocks to be mapped as linear memory must be passed to the DPMS_M_MAP function. The structure used for this is the VDS scatter/gather lock DMA Descriptor Structure (DDS) using physical addresses, not the VDS simple lock DDS which can only describe one physical block. The structure of the VDS scatter/gather lock DDS using physical addresses is defined in DPMS.INC as follows:

DDS_Scatter_Gather_Struc	struc	
DDS_Size	dd ?	Total buffer size in bytes
DDS_Offset	dd ?	Offset
DDS_Segment_Selector	dw ? dw ?	Segment or selector Reserved
DDS_Entries_Available	dw ?	Maximum number of physical blocks this structure can describe
DDS_Entries_Used	dw ?	Number of physical blocks
DDS_Physical_Address	dd ?	Physical address of first physical block
DDS_Physical_Size	dd ?	Size in bytes of first physical block
DDS_Scatter_Gather_Struc	ends	

The DDS_Physical_Address and DDS_Physical_Size fields are duplicated as many times as necessary to describe all the physical blocks. The client does not usually know the actual number of physical blocks and must allow for several. The number it allows for is passed to VDS in the DDS_Entries_Available field.

The client fills in the DDS_Size, DDS_Offset, DDS_Segment_Selector and DDS_Entries_Available fields before calling the VDS scatter/gather lock function. This is done by executing an INT 04Bh with AX set to 08105h, DX set to 0 and ES:DI pointing to the DDS, although clients should first verify that VDS is present by ensuring that bit 5 of 040:07Bh is set. VDS fills in the DDS_Entries_Used, DDS_Physical_Address and DDS_Physical_Size fields, or returns an error if there are more physical blocks than the DDS allows for.

If VDS is not present, then the client must obtain the number, addresses and sizes of the physical memory blocks some other way. One possibility is that it could assume that the memory it has is in one physical block and that the address it has is the physical address, in which case it should fill in the DDS_Physical_Address and DDS_Physical_Size fields and set DDS_Entries_Used to 1.

The DPMS_M_MAP function only requires the DDS_Entries_Used, DDS_Physical_Address, and DDS_Physical_Size fields.

The DDS is assumed to be correct and its validity is not checked.

Do not make repeated DPMS_M_MAP calls without matching DPMS_M_UNMAP calls (see page 2-60), because each DPMS_M_MAP call consumes resources.

DPMS_M_UNMAP (0304h)

Unmap As Linear Memory

Function

This optional function unmaps physical memory mapped via DPMS_M_MAP (see page 2-57). It is called via real or protected mode API code.

Entry Parameters

Register	Value
AX	DPMS_M_UNMAP
SI,DI	Handle

Returned Values

Register	Value
If the function is successful	
CF	Clear
AX	Unmodified
If the function is unsuccessful	
CF	Set
AX	Error code (see "Error Codes" on page A-1) including DPMS_E_NOT_SUPP (Unsupported function) DPMS_E_BAD_HANDLE (Invalid handle)

Explanation

In some implementations of DPMS this function is not supported. Ignore the error code "Unsupported function" that is returned in these cases.

If the client called the Virtual DMA Services (VDS) scatter/gather lock function in order to obtain physical addresses before calling DPMS_M_MAP to map the memory, then it must call the VDS scatter/gather unlock function after calling DPMS_M_UNMAP. This is done by executing an INT 04Bh with AX set to 08106h, DX set to 0 and ES:DI pointing to the DMA Descriptor Structure (DDS) that was used for the lock call.

DPMS_M_GET_PAGE_TABLE (0305h)

Get Page Table Entries

Function

This optional function gets page table entries. It is called via protected or real mode API code. This function is only supported on 386 and later processors.

Entry Parameters

Register	Value
AX	DPMS_M_GET_PAGE_TABLE
ESI	Linear memory address
(E)CX	Number of page table entries
ES: (E)DI	Pointer to buffer

Returned Values

Register	Value
If the function is successful	
CF	Clear
AX	Unmodified
ES: (E)DI	Buffer filled in with required page table entries
If the function is unsuccessful	
CF	Set
AX	Error code (see "Error Codes" on page A-1) including DPMS_E_NOT_SUPP (Unsupported function) DPMS_E_BAD_LIN_ADDR (Invalid linear address)

The available bits in each page table entry are reserved for use by the server and may contain any values when copied to the client.

DPMS_M_SET_PAGE_TABLE (0306h)

Set Page Table Entries

Function

This optional function sets page table entries. It is called via protected or real mode API code. This function is only supported on 386 and later processors.

Entry Parameters

Register	Value
AX	DPMS_M_SET_PAGE_TABLE
EBX	Linear memory handle
(E)CX	Number of page table entries
ESI	Linear address
ES: (E)DI	Pointer to page table entries

Returned Values

Register	Value
If the function is successful	
CF	Clear

AX	Unmodified
If the function is unsuccessful	
CF	Set
AX	Error code (see "Error Codes" on page A-1) including DPMS_E_NOT_SUPP (Unsupported function) DPMS_E_BAD_LIN_ADDR (Invalid linear address) DPMS_E_BAD_HANDLE (Invalid handle)

The available bits in each page table entry are reserved for use by the server and will be ignored when this function is called.

DPMS_M_MAP_SIZE (0307h)

Get Largest Mappable Block Size

Function

This optional function gets the size of the largest block of linear memory that can be mapped into DPMS linear address space. It is called via real or protected mode API code.

Entry Parameters

Register	Value
AX	DPMS_M_MAP_SIZE

Returned Values

Register	Value
If the function was successful	
CF	Clear
AX	Unmodified
BX CX	Size in bytes of largest block that can be mapped
If the function was unsuccessful	
CF	Set
AX	Error code (see "Error Codes" on page A-1) including DPMS_E_NOT_SUPP (Unsupported function) DPMS_E_LIN_NA (Linear memory unavailable)

Explanation

The returned value is 32 bits. The high word is returned in BX, the low word is returned in CX.

Servers might have limited resources for mapping physical memory as linear memory. A typical server can allocate a fixed amount of space for page tables at initialization time, and this space can be used or fragmented as memory is mapped and unmapped, thus reducing the amount of contiguous memory space available. This function can be used to get the size of the largest block that can be mapped using the Map as Linear Memory function, DPMS_M_MAP, described on page 2-57.

DPMS_M_RELOC (0400h)

Relocate Segment to Extended Memory

Function

This function allocates extended memory and relocates a real mode segment to it. It is called via real mode API code.

Entry Parameters

Register	Value
AX	DPMS_M_RELOC
BL	Descriptor type or attributes (byte 5 of the descriptor)
BH	Descriptor 386 extended type or attributes (byte 6 of the descriptor)
CX	Segment limit (size-1)
DX	Selector to use or 0 to allocate a descriptor
ES:SI	Real mode segment to be relocated

Returned Values

Register	Value
If the function is successful	
CF	Clear
AX	Selector
BX,CX	Linear address of the protected mode segment
SI,DI	Handle for the linear memory allocated
If the function is unsuccessful	
CF	Set

AX	Error code (see "Error Codes" on page A-1) including DPMS_E_DESC_NA (Descriptor unavailable) DPMS_E_LIN_NA (Linear memory unavailable) DPMS_E_PHYS_NA (Physical memory unavailable) DPMS_E_BAD_VALUE (Invalid value) DPMS_E_BAD_SEL (Invalid selector)

Explanation

The returned address and handle are 32 bits. The high words are returned in BX and SI, and the low words are returned in CX and DI.

Use this function to load a device driver or TSR, or its tables into extended memory.

The client can either call this function to move the entire driver into protected mode and then use the DPMS_CALL_PROT (see page 2-18) to run the initialization code in protected mode, or it can initialize in real mode, and finally call DPMS_M_RELOC to move the initialized code and data into protected mode.

You can pass a selector obtained via DPMS_D_ALLOC (see page 2-39) to this function, or a value of zero to allocate automatically a new descriptor pointing to the new memory block.

Memory allocated via the DPMS_M_RELOC function should be freed before freeing the descriptor.

info@caldera.com
Copyright © 1994, 1997 Caldera, Inc. All rights reserved.

Chapter 3 Debugging DPMS Clients

Using WDEB386.EXE to Debug DPMS Clients Currently the recommended debugger is the Microsoft* WDEB386.EXE, which is available in the MS Windows SDK and DDK.

WDEB386.EXE requires a 386* processor, because it uses hardware features such as the debug registers. It also requires a serial terminal.

To debug clients of Caldera's DPMS server, do not use the standalone DPMS server (DPMS.EXE), but rather the combined EMM386/DPMS driver (EMM386.EXE) which has been created specifically to enable debugging of clients. This is a fully functional EMM386 that supports expanded and extended memory, and should be used in place of any other version of EMM386 you normally use.

Load the programs in this order

1. WDEB386.EXE

2. EMM386.EXE

3. Your DPMS client

You can place INT 1 break points into your program at build time, or press <Ctrl> C on the serial terminal at run time to break into WDEB386. The advantage of the combined EMM386/DPMS server is that it allows you to examine selectors, extended memory and descriptor tables from a break point in either the real mode or the protected mode section of your program.

Apart from deliberate break points, your program may cause a processor exception, which the DPMS server will trap. Some of these are part of normal operation and the DPMS server handles them. Others, which imply an error, are passed on to WDEB386 in a manner that simulates a break point in your program, so you can see the exact instruction that caused the exception.

You can use all the features of WDEB386 to debug your client except those that are Windows-specific. You cannot debug DPMS clients while Windows is running.

---

info@caldera.com
Copyright © 1994, 1997 Caldera, Inc Corporation. All rights reserved.

Appendix A DPMS Error Codes

Error Codes

All these error codes currently match the DPMI error codes. The following equates are in DPMS.INC.

Error	Error Code	Explanation
DPMS_E_GENERAL	8000h	General error. An undefined error has occurred.
DPMS_E_NOT_SUPP	8001h	Unsupported function. The DPMS function is not defined, is currently disabled, or the function is not supported by the server.
DPMS_E_PROTECTED	8002h	Cannot switch to protected mode. The CPU is in protected mode and DPMS cannot get control.
DPMS_E_NO_DEF_STACK	8004h	No default stack is defined. The client registered with a default stack size of zero, but has requested a default stack.
DPMS_E_UNKNOWN_CLIENT	8005h	Unknown client. The client ID in the client's API code is invalid.
DPMS_E_RESOURCE_NA	8010h	Resource unavailable. A resource for which there is no specific error code, for example, internal table space, is unavailable.
DPMS_E_DESC_NA	8011h	Descriptor unavailable. There are insufficient free descriptors available.
DPMS_E_LIN_NA	8012h	Linear memory unavailable. There is insufficient free linear memory mapping space available.
DPMS_E_PHYS_NA	8013h	Physical memory unavailable. There is insufficient free physical memory available.
DPMS_E_BAD_VALUE	8021h	Invalid value. An invalid value was passed as a parameter.
DPMS_E_BAD_SEL	8022h	Invalid selector. An invalid selector was passed as a parameter.
DPMS_E_BAD_HANDLE	8023h	Invalid handle. An invalid handle was passed as a parameter.
DPMS_E_BAD_LIN_ADDR	8025h	Invalid linear address. An invalid linear address was passed as a parameter.

info@caldera.com
Copyright © 1994, 1997 Caldera, Inc. All rights reserved.